

UF3

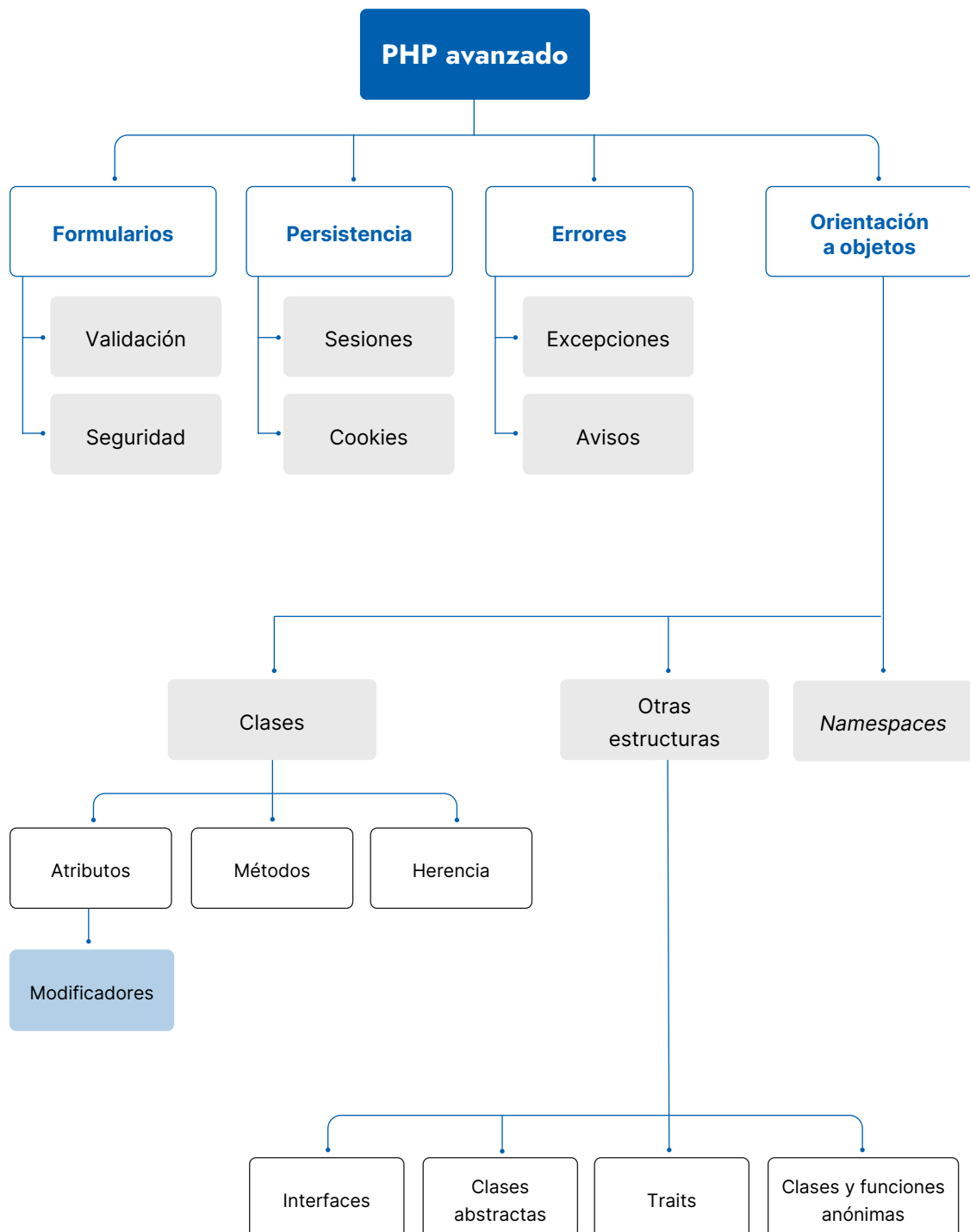
PHP Avanzado

Desarrollo web en entorno servidor

ÍNDICE

Mapa conceptual.....	03
01 Formularios	04
1.1. Formularios en HTML	04
1.2. Recepción de la información. <i>GET/POST</i>	05
1.3. Acceso a diferentes tipos de elementos de un formulario	06
02 Validación de datos. Seguridad	09
2.1. Validación de formularios	09
2.2. Seguridad	12
A. Inyección de código en el formulario	12
B. Suplantación del valor de una variable	14
03 Sesiones y <i>cookies</i>.....	18
3.1. Sesiones.....	18
3.2. <i>Cookies</i>	22
04 Validación de datos. Seguridad	23
4.1. Excepciones.....	23
4.2. Otros errores y notificaciones	28
05 Orientación a objetos con PHP.....	31
5.1. Clases, atributos y métodos en PHP	32
5.2. Creación y uso de objetos	33
5.3. Métodos constructores y destructores.....	34
5.4. Herencia.....	36
5.5. Los modificadores final, const y static	37

Mapa conceptual



01 Formularios

La interacción con los usuarios es básica para cualquier aplicación, y también lo es, incluso más, en una aplicación web. La principal herramienta de que disponen los usuarios para comunicar información a la aplicación es el formulario.

Puedes acceder a los ficheros con todos los ejemplos de este apartado descargando el archivo `DWES_U03_A01.zip`, en la sección Ejemplos para trabajar y analizar de la unidad.

EJEMPLO

1.1. Formularios en HTML

Como ya sabrás, en HTML disponemos de la sección `<form>...</form>` para definir un formulario. Entre la etiqueta de apertura y la de cierre podemos incluir diferentes elementos del lenguaje HTML, principalmente elementos interactivos, como cajas de texto, selectores de diversos tipos o botones. No es el momento de repasar todos ellos, pero sí de que recordemos lo más importante de un formulario para el tema que nos incumbe:

- » El atributo `action` en la etiqueta de apertura del formulario `<form>` define a qué página se enviarán los datos del formulario.
- » El atributo `method` en la misma etiqueta `<form>` especifica el método de envío que se usará para pasar los datos: `GET` o `POST`.
- » El botón de tipo `submit` lanza el envío de datos a la página que se haya designado como receptora de estos.

Así, un formulario básico en HTML puede tener la siguiente forma:

```
<!doctype html>
<html>
<head>
<meta charset="UTF-8">
<title>Ejemplo 01</title>
</head>
<body>
<h1>Mi formulario</h1>
<body>
<form action="ejemplo01.php" method="POST">
```

```
Nombre: <input type="text" name="nombre"/><br />
Apellidos: <input type="text" name="apellidos"/><br />
<input type="submit" value="Enviar">
</form>
</body>
</body>
</html>
```

El valor del atributo `action` es `ejemplo01.php`, lo que significa que la información introducida en el formulario será enviada a ese documento cuando se pulse el botón **Enviar**, ya que este botón tiene asignado el tipo `submit`.

Y una vez llega la información a `ejemplo01.php`, ¿cómo trabajamos con ella? Eso es lo que veremos en el siguiente punto.

1.2. Recepción de la información. *GET/POST*

La forma de acceder a la información desde el *script* de destino es muy sencilla. Si el método usado en el formulario (atributo `method`) es `GET`, entonces la información está disponible en el *array* asociativo `$_GET`; si, por el contrario, el método usado es `POST`, podremos acceder a la información a través de `$_POST`.

IMPORTANTE

En caso de no estar seguros de si la información llega a la página como `POST` o como `GET`, siempre podemos usar el *array* asociativo `$_REQUEST`, que sirve para ambos casos.

Supongamos que el formulario de ejemplo que hemos visto está en una página HTML denominada `ejemplo01.html`. Si vamos a esa página, rellenamos el formulario y pulsamos el botón **Enviar**, los datos llegarán a la página `ejemplo01.php`, donde podemos hacer que se muestren por pantalla. El contenido de `ejemplo01.php` puede ser tan sencillo como:

```
<?php
echo "Tu nombre completo es " . $_POST['nombre'] . " " . $_POST["apellidos"] ;
?>
```

Así, al lanzar el formulario, vemos:

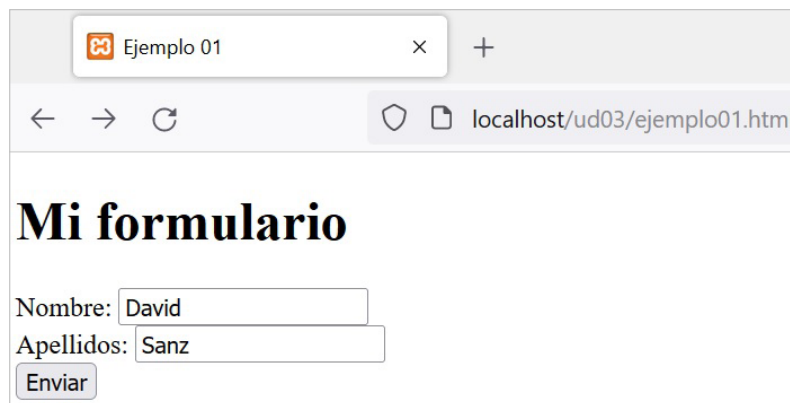


Figura 1.

Y al pulsar el botón **Enviar**:

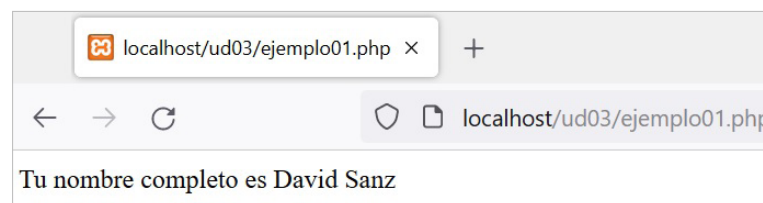


Figura 2.

1.3. Acceso a diferentes tipos de elementos de un formulario

Como hemos visto, el acceso a los datos del formulario se produce mediante el acceso a los arrays asociativos `$_GET` o `$_POST`, según el método de envío utilizado. Eso es así de sencillo para la mayoría de datos que se pueden enviar, como elementos **INPUT** de tipo **text**, **number** o **password**. También lo es para los elementos de tipo **checkbox**, con la salvedad de que habrá que comprobar, como haremos después, si el elemento en cuestión está marcado o no. Los casos que debemos tener en cuenta son los elementos **INPUT** de tipo **radio** y los de tipo **SELECT...OPTION** cuando puede seleccionarse más de un elemento (opción **multiple** activada) o **checkbox** (cuando tengamos varias opciones relacionadas con el mismo nombre).

En el caso de un **input** de tipo **radio**, si está agrupado junto a otros **inputs** del mismo tipo con el mismo nombre, usaremos el nombre común a todos los inputs como clave del **array GET/POST**, y nos devolverá el atributo **value** del **radiobutton** seleccionado. Por ejemplo, si añadimos a nuestro formulario las líneas

```
<label>Deseo ser atendido/a en: </label>
<input type="radio" name="idioma" value="es"/>Castellano
<input type="radio" name="idioma" value="en"/>English<br/>
```

en la página de destino podríamos determinar qué idioma se ha escogido comprobando el valor de `$_POST["idioma"]` y adaptar así el mensaje de bienvenida:

```
<?php
$idioma=$_POST['idioma'];
if($idioma=="es") {
    $bienvenida="Tu nombre completo es ";
}
elseif($idioma=="en") {
    $bienvenida="Your full name is ";
}
echo $bienvenida." ".$_POST['nombre']." ".$_POST["apellidos"];
?>
```

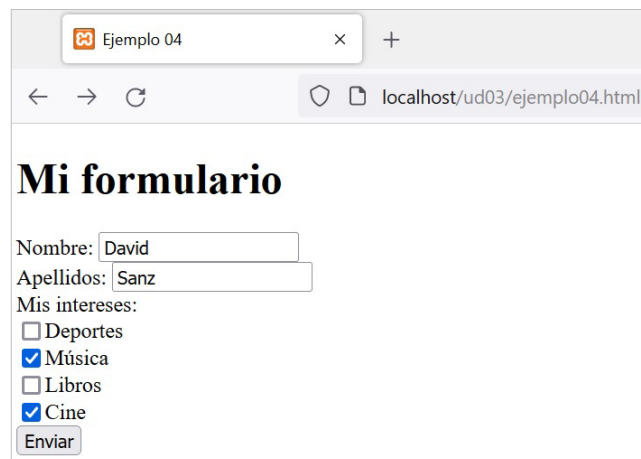
En los casos en que se puedan seleccionar varios valores con un mismo nombre, la página de destino recibirá un *array* asociativo con los valores seleccionados. Veamos un ejemplo de **checkbox**:

```
<form action="ejemplo04.php" method="POST">
Nombre: <input type="text" name="nombre"/><br />
Apellidos: <input type="text" name="apellidos"/><br />
<label>Mis intereses: </label><br/>
<input type="checkbox" name="interes[]" value="Deportes"/>Deportes<br/>
<input type="checkbox" name="interes[]" value="Música"/>Música<br/>
<input type="checkbox" name="interes[]" value="Libros"/>Libros<br/>
<input type="checkbox" name="interes[]" value="Cine"/>Cine<br/>
<input type="submit" value="Enviar">
</form>
```

Todos los elementos de tipo **checkbox** tienen el mismo nombre, «interés», y por eso se añaden los corchetes a continuación. En la página de destino, el contenido de `$_POST["interes"]` será un *array* con los valores seleccionados. Con un bucle podremos mostrarlos por pantalla.

```
<?php
echo "Hola, ".$_POST['nombre']." ".$_POST["apellidos"]."<br/>";
echo "Tus intereses son: <br/>";
foreach($_POST['interes'] as $valor) {
    echo "- ".$valor."<br/>";
}
?>
```

Al mandar el formulario con estos datos:



Mi formulario

Nombre:

Apellidos:

Mis intereses:

☐ Deportes

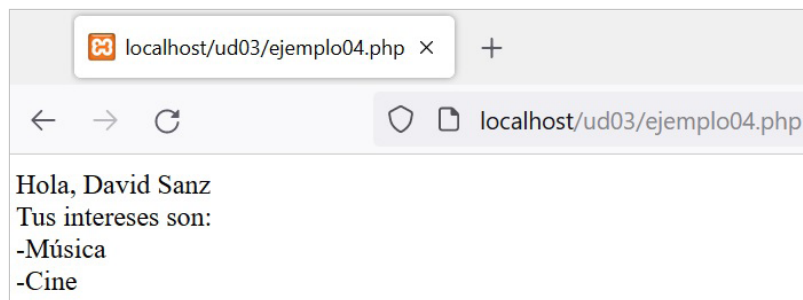
☒ Música

☐ Libros

☒ Cine

Figura 3.

obtendremos como resultado:



Hola, David Sanz

Tus intereses son:

- Música
- Cine

Figura 4.

Exactamente el mismo resultado, sin cambiar nada en la página de destino, obtendremos si cambiamos los **checkbox** por un **select...option multiple**.

```
<label>Mis intereses: </label><br/>
<select name="interes[]" size="4" multiple>
  <option value="Deportes">Deportes</option>
  <option value="Música">Música</option>
  <option value="Libros">Libros</option>
  <option value="Cine">Cine</option>
</select>
```


02 Validación de datos. Seguridad

2.1. Validación de formularios

En primer lugar, hay que recordar que HTML tiene sus propios mecanismos de validación para formularios, y es conveniente usarlos para descargar de comprobaciones sencillas a la aplicación. Por ejemplo, es posible comprobar desde el propio formulario que un dato introducido en un *input* sea un número, o una fecha válida, o una cantidad dentro de un rango, un correo electrónico o una cadena de caracteres con una longitud determinada. Esas comprobaciones se harían en el cliente, antes de enviar la información al servidor.

EJEMPLO

Puedes acceder a los ficheros con todos los ejemplos de este apartado descargando el archivo `DWES_U03_A02.zip`, en la sección Ejemplos para trabajar y analizar de la unidad.

Incluso así, es importante validar los datos en el servidor para evitar errores e inconsistencias. Recordemos que disponemos de funciones como `is_numeric()` para comprobar que una variable contenga un número, `is_integer()`, `is_real()`, `is_double()`, `is_string()`, etcétera. Para comprobar si la estructura de una cadena se ajusta a lo esperado, podemos usar expresiones regulares, tanto en el propio formulario (atributo `pattern` de la etiqueta `input`) como en PHP, usando la función `preg_match()`. Dada la gran cantidad de opciones de que disponemos en PHP para analizar una cadena usando expresiones regulares, dejamos un enlace en el material complementario para poder profundizar en su uso. No obstante, vamos a ver un ejemplo en el que comprobaremos que un DNI tiene 8 caracteres numéricos y una letra. Para ello usamos el patrón `/^[0-9]{8}[A-Z]{1}$/` , que significa que la cadena debe empezar (^) con una serie de ocho números y a continuación debe incluir una letra mayúscula, con la que acabará (\$) la cadena.

A la recepción del DNI haríamos:

```
<?php
$dni=$_POST['dni'];
$patron=' /^[0-9]{8}[A-Z]{1}$/ ';
if(preg_match($patron,$dni)) {
    echo "Tu DNI $dni es válido";
}
else {
```

```

        echo "Tu DNI $dni no tiene un formato válido";
    }
    ?>

```

Pero ¿qué ocurre si el DNI se deja en blanco, o incluso si no se llega a enviar? Para ello disponemos de otras funciones de validación:

FUNCIÓN	DESCRIPCIÓN
<code>isset(\$variable)</code>	Devuelve true si la variable existe y tiene un valor distinto de NULL
<code>is_null(\$variable)</code>	Devuelve true si la variable tiene valor NULL
<code>empty(\$variable)</code>	Devuelve true si la variable tiene un valor vacío o nulo

Volviendo a nuestro ejemplo anterior, supongamos que el usuario accede directamente a la página de validación sin haber pasado por el formulario. Verá algo así:

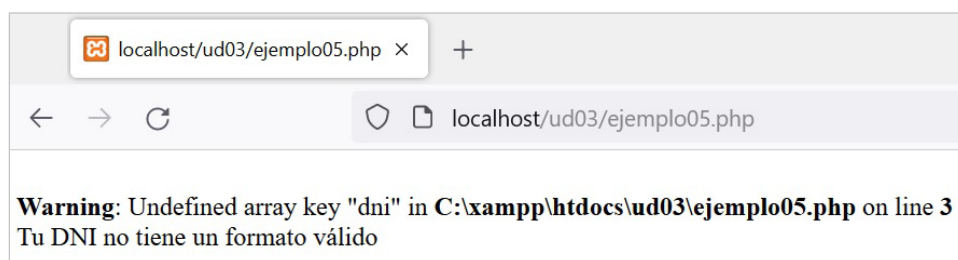


Figura 5.

Aunque efectivamente el programa nos dice que el DNI no es válido, estos mensajes y avisos pueden confundir al usuario. Es mejor comprobar que el dato se ha enviado usando alguna de las funciones anteriores. Por ejemplo:

```

<?php
if(!empty($_POST['nombre']) && !empty($_POST['apellidos']) && !empty($_POST['dni'])) {
    echo "Hola, ".$_POST['nombre']." ".$_POST['apellidos']."<br/>";
    $dni=$_POST['dni'];
    $patron='/^[0-9]{8}[A-Z]{1}$/';
    if(preg_match($patron,$dni)) { echo "Tu DNI $dni es válido"; }
    else { echo "Tu DNI $dni no tiene un formato válido"; }
}
else { echo "Debes introducir el DNI para poderlo validar"; }
?>

```

Si tratamos de acceder a la página de validación (**ejemplo05.php**) sin haber pasado por el formulario (**ejemplo05.html**), obtendremos el mensaje:

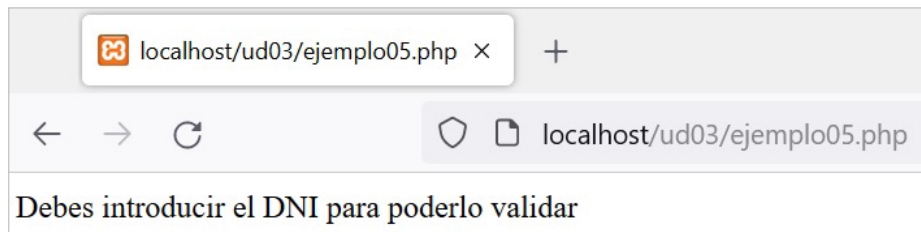


Figura 6.

IMPORTANTE

Si no queremos comprobar uno a uno todos los *inputs* del formulario para comprobar si el usuario ha pasado por él o no, en realidad sería suficiente con comprobar con **isset()** uno cualquiera de ellos. Si la función devuelve **false**, el usuario se ha saltado el formulario.

En nuestro ejemplo, simplemente mostramos un mensaje de error o aviso, pero es una buena práctica que la comprobación de los datos del formulario se haga en una página intermedia que luego redirija la ejecución de la aplicación a la página de destino (si los datos son correctos) o de nuevo al formulario (si no lo son). En ocasiones, es la propia página donde se encuentra el formulario la que hace la función de validación. En cualquier caso, para forzar el salto a una página determinada necesitamos la función **header**. Su formato es:

```
header("Location:nombre_página");
```

Puesto que **nombre_página** es en realidad una URL, podemos pasarle datos a la página invocada, tal como hemos visto que hace un formulario con **method GET**:

```
header("Location:nombre_página?variable=valor");
```

IMPORTANTE

La instrucción **header** debe situarse antes de que se muestre cualquier información por pantalla. De hecho, es recomendable que la página que distribuye la ejecución con **header** no muestre nada por pantalla ni realice ninguna interacción con el usuario.

Otra forma de pasar información desde una página a otra, sin intervención directa del usuario, es añadiendo campos de tipo `hidden` a nuestros formularios. Podemos pasar el valor que deseemos a la página de destino, que lo leerá igualmente como parámetro GET o POST.

En próximos apartados veremos cómo controlar que el usuario no pueda acceder a la página de destino sin haber pasado por el proceso de validación.

2.2. Seguridad

Como hemos comentado anteriormente, la interacción con los usuarios hace que la seguridad de nuestra aplicación empiece a ser una prioridad. Hay numerosos aspectos que podríamos tener en cuenta, pero aquí nos centraremos, de momento, en la seguridad relacionada con la introducción de datos a la aplicación.

A. Inyección de código en el formulario

Un agujero típico de seguridad en las aplicaciones web es el que permite los ataques XSS (*cross-site scripting*), en los que un usuario puede introducir en la página web un código externo a la propia aplicación y hacer que se ejecute. Mediante esta ejecución de código externo, se puede comprometer toda la integridad del sistema, robar datos, introducir virus o cualquier otra acción maliciosa.

El ataque XSS puede ser directo o indirecto. En el primer caso se introduce, generalmente en un *input* de un formulario, el código malicioso de forma directa. Si dicha información no es filtrada correctamente antes de su utilización, puede desencadenar la temida ejecución de código externo. En el segundo caso, el XSS indirecto, lo que se hace es modificar el valor de alguna variable usada por el sistema. En el siguiente punto veremos cómo combatir esta opción.

Veamos un ejemplo de ataque directo. Supongamos que tenemos un formulario que pide el nombre del usuario y lo lanza directamente a otra página donde se muestra por pantalla.

```
<form action="ejemplo08destino.php"
method="POST">
Nombre: <input type="text"
name="nombre"/><br />
Apellidos: <input type="text"
name="apellidos"/><br />
<input type="submit" value="Enviar">
</form>
```

ejemplo08.php

```
<?php
echo "Hola, tu nombre es ";
echo $_POST['nombre'];
echo " y tus apellidos ";
echo $_POST['apellidos'];
?>
```

ejemplo08destino.php

Esquema 1.

Si introducimos el nombre y los apellidos en el formulario se mostrarán por pantalla. ¿Qué ocurre si, en lugar de nuestro nombre, introducimos un código como, por ejemplo, el siguiente?

```
<script>alert("Tu web acaba de ser hackeada")</script>
```

En la página de destino, al intentar mostrar por pantalla el nombre introducido por el usuario, lo que se verá realmente es:

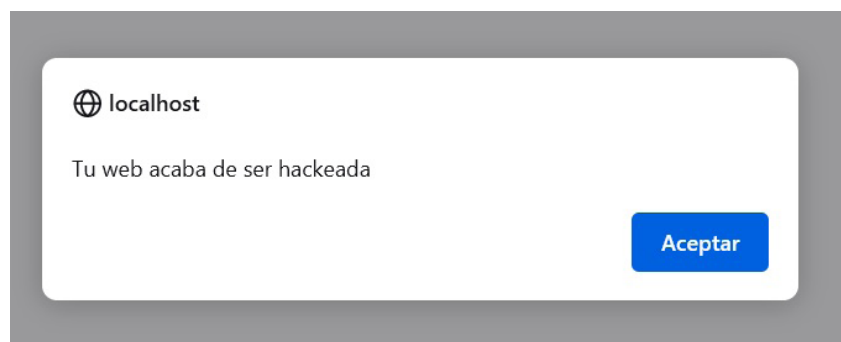


Figura 7.

En este caso se trata solo de una alerta, pero podríamos haber escrito cualquier código en javascript mucho más peligroso y se hubiera ejecutado igualmente.

La solución a este tipo de ataques está en la validación de los datos introducidos en el formulario. Como ya sabemos, dicha validación puede realizarse en el propio formulario o en la página de destino. Tenemos diferentes opciones:

- » Validación en el propio formulario usando el **type** correspondiente y patrones de cadena (la menos segura; es importante validar también en el servidor).
- » Validación en la página de destino usando patrones de cadena y **preg_match()**.

```
$nombre=$_POST['nombre'];  
$patron='/[A-Za-z]$/' ;  
if(preg_match($patron,$nombre)) {  
    echo "Hola, tu nombre es ";  
    echo $_POST['nombre'];  
}  
else { echo "Nombre no válido"; }
```

» Usar filtros predefinidos y funciones de filtrado (ver enlace en el material complementario).

» Usar funciones de conversión como `strip_tags` o `htmlspecialchars`

```
$nombre01=strip_tags($_POST['nombre']);  
$nombre02=htmlspecialchars($_POST['nombre']);  
echo "Nombre saneado con strip_tags: ";  
echo $nombre01;  
echo "<br/>";  
echo "Nombre saneado con htmlspecialchars: ";  
echo $nombre02;
```

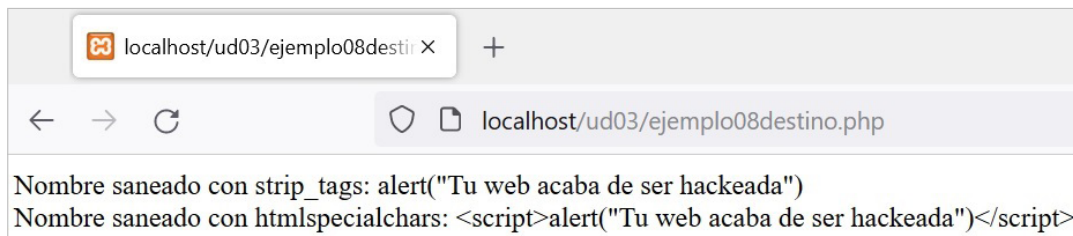


Figura 8.

B. Suplantación del valor de una variable

La inyección de código puede ser indirecta, suplantando el valor de una variable pasada como parámetro a una página.

Puesto que de momento solo sabemos pasar una variable a una página a través del formulario (caso expuesto en el punto anterior) o enganchando la variable a la URL (equivalente al método `GET` de un formulario), sería suficiente con validar y sanear las variables que llegan por `$_GET` de la misma forma que hemos visto en el punto anterior.

Supongamos que tenemos nuestro formulario convenientemente saneado en HTML pero la información se pasa por `GET`. Dicha información, como sabemos, queda expuesta en el navegador:



Esquema 2.

Así, cualquier usuario podría realizar una sustitución de variable directamente en la URL:

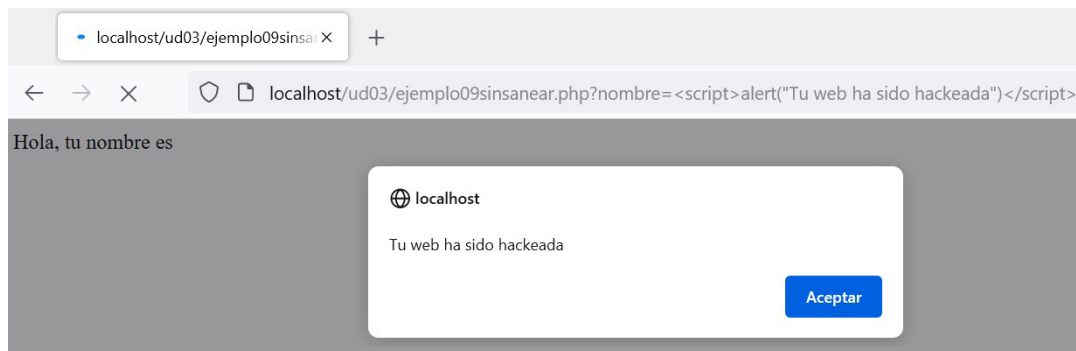


Figura 10.

La solución es sanear la información recibida por `$_GET` igual que hemos hecho anteriormente con la información que recibimos por `$_POST`.

```
<?php
if(isset($_GET['nombre'])) {
    $nombre01=strip_tags($_GET['nombre']);
    $nombre02=htmlspecialchars($_GET['nombre']);
    echo "Nombre saneado con strip_tags: ";
    echo $nombre01;
    echo "<br/>";
    echo "Nombre saneado con htmlspecialchars: ";
    echo $nombre02;
}
else { echo "No has introducido tu nombre"; }
?>
```

Ahora, si tratamos de insertar código en la URL a través del parámetro `$_GET['nombre']`:

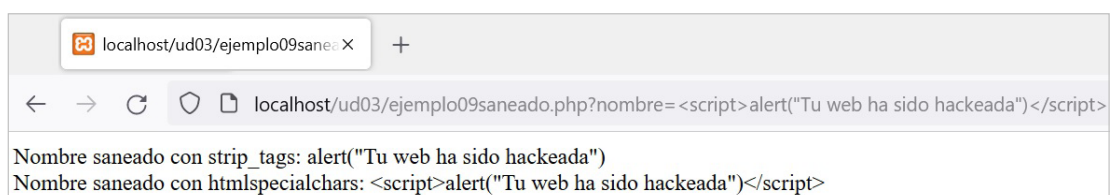


Figura 11.

1

CASO PRÁCTICO



VALIDAR UN FORMULARIO

Vamos a diseñar un formulario que le pida al usuario una serie de datos personales: nombre y apellidos, dirección, población y sexo (masculino/femenino). El formulario incluirá un **checkbox** que el usuario deberá obligatoriamente marcar indicando que acepta las condiciones de acceso de nuestra página. Un botón de enviar mandará la información a una página de destino, donde la información debe validarse, comprobando que todos los *inputs* se han rellenado, que el **checkbox** de aceptación de condiciones se ha marcado, y que la población sigue el patrón de cinco números (código postal), un guion y el nombre de la población, todo ello sin espacios en blanco. Es decir, **28001-Madrid** sería una población válida.

En caso de no cumplirse alguna de las condiciones, o si se intenta acceder a la página de destino sin haber pasado por el formulario, se redirigirá automáticamente al usuario a la página inicial para que vuelva a validarse, mostrando un mensaje que especifique el error detectado. Si toda la información es correcta y se han aceptado las condiciones de la web, la página de destino mostrará un mensaje de bienvenida («Bienvenido» para sexo masculino y «Bienvenida» para sexo femenino), seguido del nombre completo introducido por el usuario.

SOLUCIÓN

En el formulario evitamos usar **required**, para así validar que se rellenan todos los *inputs* en el *script* de destino. Los *inputs* para el nombre, dirección y población serán de tipo **text**. El *input* para el sexo masculino/femenino será de tipo **radio**, mientras que el *input* de aceptación de las condiciones será, como se ha comentado, un **checkbox**. El formulario comprobará si se ha recibido un mensaje de error a través de **\$_GET**, y, si es así, lo mostrará por pantalla.

Puedes encontrar los códigos completos en los ficheros incluidos en el archivo **DWES_U03_CP_A02_SOL.zip**, en la parte izquierda de la pantalla.

Esta primera página con el formulario podría quedar así (fichero **u03_a02_cp.php**):


```

<!doctype html>
<html>
<head>
<meta charset="UTF-8">
<title>Caso práctico Unidad 03 Apartado 02</title>
</head>
<body>
<h3>Validación datos usuario</h3>
<body>
<?php
if(isset($_GET['mensaje'])) {
    $mensaje=strip_tags($_GET['mensaje']);
    echo "<h4>$mensaje</h4>";
}
?>
<form action="u03_a02_cp_destino.php" method="POST">
Nombre: <input type="text" name="nombre"/><br />
Apellidos: <input type="text" name="apellidos"/><br />
Dirección: <input type="text" name="direccion"/><br />
Población: <input type="text" name="poblacion"/><br />
Género: Masculino <input type="radio" name="genero" value="masculino"/>
Femenino <input type="radio" name="genero" value="femenino"/><br/>
He leído y acepto las condiciones de la página web
<input type="checkbox" name="acepto"/><br/>
<input type="submit" value="Enviar">
</form>
</body>
</html>

```

Como se ve, se comprueba si se ha recibido algún mensaje por **\$_GET**, y, si es así, se sanea y se muestra por pantalla. La información del formulario se manda a la página de destino, en la que, en primer lugar, comprobamos si hemos recibido datos desde el formulario; a continuación, si se han rellenado todos (incluyendo el **checkbox** de aceptación), y, finalmente, si la población cumple con el patrón establecido.

Cualquier error deriva a la página principal para que se realice de nuevo la validación mandando un mensaje por **\$_GET**, mientras que, si todo es correcto, se muestra el mensaje de bienvenida. El código de esta segunda página quedaría así (fichero **u03_a02_cp_destino.php**):

```

<?php
if(!isset($_POST['nombre'])) {
    // comprueba que se han mandado datos
    $mensaje="Debes rellenar el formulario";
    header("location:u03_a02_cp.php?mensaje=$mensaje");
}
elseif(empty($_POST['nombre']) ||
    empty($_POST['apellidos']) ||
    empty($_POST['direccion']) ||
    empty($_POST['poblacion']) ||
    empty($_POST['genero']) ||

```

```
is_null($_POST['genero'])) {  
    // comprueba que se han rellenado los inputs  
    $mensaje="Debes rellenar todos los campos del formulario";  
    header("location:u03_a02_cp.php?mensaje=$mensaje");  
}  
elseif(empty($_POST['acepto'])) {  
    // comprueba que se han aceptado las condiciones  
    $mensaje="Debes aceptar las condiciones de acceso";  
    header("location:u03_a02_cp.php?mensaje=$mensaje");  
}  
elseif(!preg_match('/^[0-9]{5}-[A-Za-z]+$/', $_POST['poblacion'])) {  
    // comprueba que la población sigue el patrón  
    $mensaje="La población no sigue el formato establecido";  
    header("location:u03_a02_cp.php?mensaje=$mensaje");  
}  
else {  
    // todo correcto, se sanean los datos y se muestran  
    $nombre=strip_tags($_POST['nombre']);  
    $apellidos=strip_tags($_POST['apellidos']);  
    $genero=strip_tags($_POST['genero']);  
    $bienvenida=($genero=="masculino"? "Bienvenido, ":"Bienvenida, ");  
    echo $bienvenida.$nombre." ".$apellidos;  
}  
?>
```

03 Sesiones y cookies

Toda la información que hemos manejado hasta ahora se guarda en la memoria del cliente y desaparece cuando acaba la ejecución de la aplicación, ya que **HTTP** es un **protocolo sin estado**: cada petición al servidor es una conexión nueva y no guarda información de una conexión a la siguiente salvo que la pasemos explícitamente de alguna de las formas que hemos ido estudiando. Ahora lo que buscamos es cierta **persistencia** de la información sin tener que acudir, de momento, a ficheros y bases de datos. Esta persistencia nos la darán las **sesiones** en tiempo de ejecución, y las **cookies** entre una ejecución y otra.

3.1. Sesiones

Muchos lenguajes de programación, incluyendo PHP, permiten manejar el concepto de **sesión**. Este concepto se refiere al control del tiempo que un usuario está interactuando con la aplicación. La sesión debe iniciarse explícitamente mediante ciertas instrucciones, y su duración máxima se puede limitar e incluso se puede forzar el final de una sesión si se

detecta inactividad por parte del usuario durante un tiempo determinado. También, y aquí es donde volvemos a la cuestión de la persistencia de los datos, podemos definir ciertas variables que permanezcan en la memoria y sean accesibles mientras dure la sesión, aunque cambiemos de unas páginas a otras.

En primer lugar, veamos cómo iniciar una sesión en PHP. Para ello llamamos a la función `session_start()`. Esta instrucción debe ejecutarse antes de que se muestre ninguna información por pantalla. Una vez iniciada la sesión, se crea automáticamente un *array* asociativo llamado `$_SESSION`, que podemos usar para guardar información y recuperarla en cualquier momento mientras la sesión esté activa. Por ejemplo, podemos guardar el nombre del usuario en una variable de sesión, y de esa forma podríamos mostrarlo en cualquier página.

```
$_SESSION["nombre"]=$nombreUsuario;
```

Las variables de sesión pueden mostrarse por pantalla; podemos consultar su valor cuando sea necesario y también modificarlo en cualquier momento. También pueden borrarse con la función `unset()`.

```
unset($_SESSION["nombre"]);
```

Si queremos borrar todas las variables de sesión sin tener que hacerlo una a una, lo podemos hacer con la función `session_destroy()`. En teoría, esto debería cerrar la sesión, pero, si internamente se han usado *cookies* para persistir la información de la sesión, hará falta borrar también la *cookie* correspondiente. El siguiente código debería eliminar la sesión y toda su información asociada.

```
<?php
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000,
        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]
    );
}
?>
```

Este código puede guardarse en una función o página y llamarse cuando queramos cerrar la sesión.

Si queremos hacer cambios en las variables de configuración sin modificar los ficheros de configuración de PHP, de forma que dichos cambios solo afecten a la sesión/aplicación actual, lo podemos hacer mediante la función `ini_set()` antes de ejecutar `session_start()`. Con la función `ini_set()` se pueden cambiar diversas directivas de PHP, no solo las relacionadas con las sesiones. En el material complementario encontrarás un enlace en el que podrás comprobar cuáles son esas directivas y sus funciones correspondientes.

La función `session_start()` debe ser llamada al principio de cada página en la que queramos trabajar con sesiones. Si no lo hacemos, el array `$_SESSION` será inaccesible.

Vamos a iniciar una sesión con un nombre de usuario predeterminado. Al pulsar sobre un enlace, cambiaremos de página y mostraremos el nombre de usuario.

Para ello tenemos una página, **ejemplo10.php**, con este contenido dentro de

`<body>...</body>`:

```
<?php
session_start();
$_SESSION["usuario"]="Mercedes";
?>
Haga clic para iniciar la sesión
```

1

Como se ve, se inicia la sesión y se crea una variable de sesión llamada `usuario`, cuyo contenido es «Mercedes». En la página llamada mediante el enlace, **ejemplo10destino.php**, tenemos:

```
<?php
session_start();
if(isset($_SESSION['usuario'])) {
    echo "Hola, ".$_SESSION['usuario'];
}
else {
    echo "Debes iniciar sesión";
}
?>
```

2

Económica

Debemos fijarnos en que lo primero que hacemos es un `session_start()` para continuar con la sesión que hemos iniciado en la primera página, o iniciar una sesión nueva en caso de que no se haya iniciado ninguna. En este segundo caso, que se produce si el usuario va directamente a la página **ejemplo10destino.php** sin pasar por la página **ejemplo10.php**, la sesión se inicia, pero la variable `usuario` no existirá. Con la función `isset` comprobamos si la variable de sesión existe o no, y, en caso de que no exista, mostramos un mensaje. Añadiendo una instrucción `header()` podríamos forzar a que el usuario se dirija a la página principal para iniciar la sesión.

¿Y qué ocurre con el tiempo de inactividad? Supongamos que queremos que la sesión se destruya si se detecta un minuto de inactividad. Para ello, podríamos guardar el momento de inicio de sesión en la página principal:

```
$_SESSION["login_time"]=time();
```

Y en la página de destino comprobar que la diferencia de segundos transcurridos desde el inicio de sesión en la página principal es menor que 60.

```
if(time()-$_SESSION['login_time'] < 60) {
    echo "Hola, ".$_SESSION['usuario'];
}
else {
    echo "1 minuto de inactividad. Inicia sesión de nuevo";
}
```

3

De nuevo, usando la instrucción `header()` podríamos forzar al usuario a redirigirse a la página principal para volver a iniciar la sesión.

Si la comprobación del tiempo transcurrido desde el anterior inicio de sesión quiere hacerse en cada página, podríamos incluir el texto anterior en un único fichero, que, con la cláusula `include`, podemos llamar en cada una de nuestras páginas.

Las variables de sesión se pueden usar asimismo para persistir información en un formulario y para comprobar que el flujo de ejecución es el correcto (el usuario se valida antes de ir a la página de destino, por ejemplo).

Puedes ver un ejemplo completo de validación de usuario mediante sesiones en el material adjunto (ficheros **ejemplo10.php**, **ejemplo10valida.php**, **ejemplo10destino.php**, **ejemplo10comprueba.php** y

ejemplo10cierra.php). Puedes comprobar qué sucede si intentas ir a cualquiera de las otras páginas sin haberte validado en **ejemplo10.php** o después de haber cerrado la sesión, si no te validas correctamente o si tardas más de un minuto en validarte.

4

Esquema 4.

3.2. Cookies

Las *cookies* (galletas) son pequeños ficheros que la aplicación web crea en el ordenador cliente, de forma que puede guardar información sobre cada conexión que se ha realizado desde cualquier máquina. Esos ficheros solo son legibles por la aplicación que los ha creado, con lo que en principio se garantiza que esa información no se comparte con otras aplicaciones. De todos modos, siempre debemos tener en cuenta que podemos deshabilitar las *cookies* en nuestro navegador, aunque entonces es posible que algunas funcionalidades de la aplicación no se realicen correctamente.

En PHP se crea una *cookie* con la función `setcookie()`, indicando como mínimo el nombre del dato que queremos guardar y su valor. Opcionalmente, pueden añadirse el tiempo de expiración (en caso contrario, expira con la sesión), si solo debe transmitirse de forma segura (con el protocolo HTTPS), si será accesible desde otros lenguajes de *scripting*, como JavaScript, o solo a través del protocolo HTTP, e incluso el subconjunto de URL o dominios para los que la *cookie* será accesible. Dispones de más información sobre la creación de *cookies* en la sección de enlaces.

```
setcookie("ultimoacceso", date("d-m-Y"));
```

guardaría en el navegador la fecha del último acceso a la aplicación.

Hay que tener en cuenta que `setcookie()` debe ejecutarse antes que cualquier otra instrucción de salida de nuestra página, incluyendo en este caso las etiquetas `<html>` y `<head>`.

Una vez creada la *cookie*, el navegador del cliente mandará su contenido en cada conexión y podrá leerse mediante el array `$_COOKIE`, tal como hemos hecho anteriormente con las variables de sesión y `$_SESSION`.

```
echo "Hola, ".$_SESSION['nombre'].", no nos visitabas desde ".$_COOKIE['ultimoacceso'];
```

Si queremos borrar una *cookie*, podemos hacerlo con `setcookie()`, indicando el nombre del dato sin incluir ningún valor.

```
setcookie("ultimoacceso");
```

Si en la misma *cookie* queremos guardar más de un dato, lo podemos hacer creando un *array*:

```
setcookie("datos[datouno]", valoruno);  
setcookie("datos[datodos]", valordos);  
setcookie("datos[datotres]", valortres);
```

Y luego podrán leerse con un bucle:

```
if (isset($_COOKIE['datos'])) {  
    foreach ($_COOKIE['datos'] as $dato => $valor) {  
        echo "El valor de la cookie ".$dato." es ".$valor<br/>";  
    }  
}
```

En los ficheros PHP adjuntos que empiezan por **ejemplo11** podrás encontrar el ejemplo desarrollado al completo.

04 Validación de datos. Seguridad

En PHP existen diferentes categorías de errores. Podemos distinguir entre los que interrumpen la ejecución del programa y los que no. Aunque puede haber otras subdivisiones, podemos considerar que los primeros, los que provocan la interrupción del programa, son las **excepciones**. El resto, los que muestran mensajes de error, pero no interrumpen la ejecución del programa, se llaman **avisos** o notificaciones (*warnings*).

EJEMPLO

Puedes acceder a los ficheros con todos los ejemplos de este apartado descargando el archivo **DWES_U03_A04.zip**, en la sección Ejemplos para trabajar y analizar de la unidad.

4.1. Excepciones

Las excepciones en PHP aparecen cuando la aplicación debe realizar cierta tarea y no es capaz de llevarla a cabo por algún motivo ni tampoco puede seguir con el flujo normal de la ejecución. Las excepciones interrumpen, por tanto, la ejecución de la aplicación, mostrando al usuario un mensaje de error.

PHP ofrece herramientas para gestionar las excepciones, consiguiendo con dicha gestión dos objetivos:

- » Evitar que se interrumpa abruptamente la ejecución del programa.
- » Evitar que se muestren por pantalla mensajes de error que el usuario no pueda entender.

Ambos objetivos se logran haciendo uso de lo que se llama captura y manejo de excepciones, básicamente usando la estructura **try...catch**. Su funcionamiento es el siguiente:

```
try {  
    // ... instrucciones del flujo normal del programa  
}  
  
catch(error) {  
    // solo entramos aquí si se produce el error  
    // ... instrucciones para el manejo del error producido  
}
```

Si todo va bien, se ejecutará el bloque de instrucciones dentro de `try`, y no el bloque `catch`, que solo lo hará si se produce un error que pueda capturar. En ese caso, podemos controlar los mensajes mostrados al usuario y hacer que el flujo de ejecución de la aplicación, en lugar de interrumpirse, pueda seguir de alguna forma o al menos se interrumpa de manera controlada.

Las excepciones capturadas por `catch` pueden ser de un tipo concreto, o bien genéricas. En cualquier caso, todas pueden capturarse usando las clases `Exception` (hasta PHP6) o `Throwable` (a partir de PHP7). Aprovechando que un mismo bloque `try` puede tener asociados varios bloques `catch`, podemos incluir ambas clases si no sabemos exactamente la versión de PHP de la que dispone el servidor.

```
try { // código funcionamiento normal }  
catch (Throwable $e){ // solo en PHP 7 }  
catch (Exception $e){ // en versiones anteriores }
```

Con esto sería suficiente para capturar cualquier tipo de error. Pongamos un ejemplo: un formulario que pide dos números y muestra el resultado de sumarlos, restarlos, multiplicarlos y dividirlos.

Formulario:

```
<form action="ejemplo12destino.php" method="POST">  
Primer número: <input type="number" name="num1" required/><br />  
Segundo número: <input type="number" name="num2" required/><br />  
<input type="submit" value="Enviar">  
</form>
```

Recepción de datos:

```
<?php  
if(isset($_POST["num1"])) {  
    $num1=$_POST["num1"];  
    $num2=$_POST["num2"];  
    echo "Número 1: $num1 <br/>";  
    echo "Número 2: $num2 <br/>";  
    echo "La suma de $num1 y $num2 es ".($num1+$num2)." <br/>";  
}
```



```

    echo "La resta de $num1 y $num2 es " . ($num1-$num2) . "<br/>";
    echo "La multiplicación de $num1 y $num2 es " . ($num1*$num2) . "<br/>";
    echo "La división de $num1 y $num2 es " . ($num1/$num2) . "<br/>";
}
else { header("location:ejemplo12.php"); }
?>

```

Usando el tipo **number** y el atributo **required** en el formulario, evitamos que se manden datos en blanco o no numéricos. Por otro lado, en la página de destino usamos la función **isset()** para comprobar que se han enviado datos desde el formulario, y en caso contrario forzamos que se vuelva a este. Así pues, tenemos controlados todos los posibles problemas..., excepto uno. ¿Qué ocurre si el segundo número es un 0?

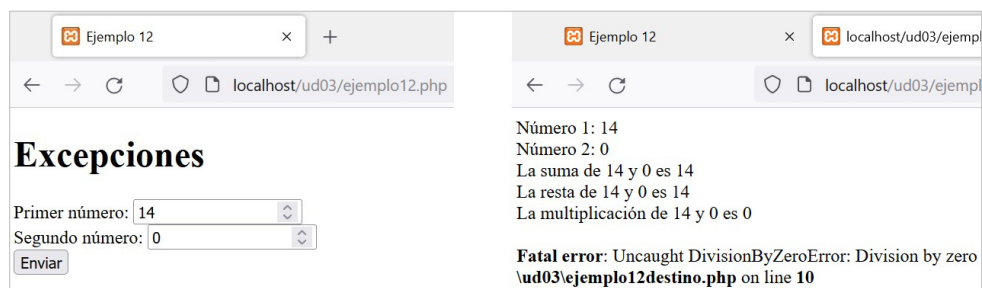


Figura 12.

Como vemos, se produce un error fatal que interrumpe la ejecución del programa. El mensaje nos dice que hemos intentado dividir por cero, algo que los lenguajes de programación no permiten. También vemos el tipo de excepción (**DivisionByZeroError**) y nos indica que no la hemos capturado (**Uncaught**). Vamos, pues, a capturarla. Además añadiremos en la página del formulario una opción para mostrar mensajes.

Formulario:

```

<?php
if(isset($_GET['message'])) {
    echo "<h3>".htmlspecialchars($_GET['message'])."</h3>";
}
?>

<form action="ejemplo13destino.php" method="POST">
Primer número: <input type="number" name="num1" required/><br />
Segundo número: <input type="number" name="num2" required/><br />
<input type="submit" value="Enviar">
</form>
</body>

```

Recepción:

```
<?php
if(isset($_POST["num1"])) {
    try {
        $num1=$_POST["num1"];
        $num2=$_POST["num2"];
        echo "Número 1: $num1 <br/>";
        echo "Número 2: $num2 <br/>";
        echo "La suma de $num1 y $num2 es ".($num1+$num2). "<br/>";
        echo "La resta de $num1 y $num2 es ".($num1-$num2). "<br/>";
        echo "La multiplicación de $num1 y $num2 es ".($num1*$num2). "<br/>";
        echo "La división de $num1 y $num2 es ".($num1/$num2). "<br/>";
    }
    catch(DivisionByZeroError $excepcion) {
        header("location:ejemplo13.php?message=No puedes dividir por 0");
    }
}
else {
    header("location:ejemplo13.php?message=Rellena el formulario");
}
?>
```

Como puede verse, hemos incluido todas las operaciones dentro de un bloque **try**. En el bloque **catch** asociado capturamos la excepción **DivisionByZeroError**, y, de producirse, reenviamos la ejecución al formulario y mostramos un mensaje indicando que el segundo número no puede ser 0.



Figura 13.

Si se produce un error del tipo especificado en **catch**, volvemos al formulario y mostramos un mensaje en lugar de interrumpir el programa. Si queremos saber cuál era el mensaje original, lo podemos conocer haciendo uso de la variable que hemos definido dentro de **catch**, **\$excepcion**, y su función asociada llamada **getMessage()**, con la que podemos

obtener el mensaje de error original. De hecho, tiene más propiedades y utilidades, como podrás ver en el enlace correspondiente que te facilitamos en el material complementario.

```
catch(DivisionByZeroError $excepcion) {  
    //header("location:ejemplo13.php?message=No puedes dividir por 0");  
    echo $excepcion->getMessage();  
}
```

En este ejemplo estamos capturando solo un posible error: la división por cero. ¿Y si se produjera otro error distinto? Pongamos por ejemplo que el formulario permite introducir caracteres no numéricos o en blanco. En ese caso se produciría otro tipo de excepción que no estaríamos capturando. Tenemos dos posibles soluciones:

- » Capturar todas las posibles excepciones con las clases **Exception** (anterior a PHP7) o **Throwable** (PHP7 y posteriores) y usar **getMessage()** para devolver el error.
- » Capturar cada excepción distinta en un **catch**, de forma que se pueda personalizar el mensaje de error.

La primera opción:

```
catch(Throwable $excepcion) {  
    header("location:ejemplo14.php?message=Error del tipo".$excepcion->getMessage());  
}
```

Siempre es mejor definir manejadores para Tipos específicos que globales.

La segunda opción:

```
catch(DivisionByZeroError $excepcion) {  
    header("location:ejemplo14.php?message=No puedes dividir por 0");  
}  
  
catch(TypeError $excepcion) {  
    header("location:ejemplo14.php?message=Los datos deben ser numéricos");  
}
```

El error **TypeError** se produce al hacer operaciones matemáticas con contenido no numérico; el error **DivisionByZeroError**, como hemos visto, si intentamos dividir por 0. De esta manera, a cada error podemos responder con un mensaje personalizado sin depender del contenido de **getMessage()**.

Las excepciones también pueden lanzarse voluntariamente, antes de que ocurran, con el comando `throw`. Es habitual hacerlo en funciones para realizar la captura en el programa principal, evitando así repetir el bucle `try...catch` en cada función.

```
<?php
function f1() {
    if (...) { throw new Exception("Mensaje1"); }
    return...
}
function f2() {
    if (...) { throw new Exception("Mensaje2"); }
    return ...
}

// con una estructura try...catch capturamos ambas excepciones
try {
    // llamada a la función f1
    // llamada a la función f2
} catch (Exception $excepcion) {
    echo "Ha ocurrido la excepción ".$excepcion->getMessage();
}
?>
```

4.2. Otros errores y notificaciones

Algunos errores no pertenecen a la categoría `Throwable` y no son capturados por `try...catch`. Es el caso de los avisos o *warnings*. Un aviso es un error no fatal, por lo que no interrumpe la ejecución de la aplicación. Así, el programa puede seguir funcionando, pero es probable que no lo haga correctamente. Por eso, aunque los avisos pueden desactivarse desde el fichero de configuración `php.ini` o con la función `error_reporting(NIVEL)`, es mejor no hacerlo, para detectar su existencia durante el desarrollo.

Una forma de capturar los avisos como si fueran excepciones es lanzar una excepción cuando se produzcan y redirigir su gestión a una función propia que podamos controlar. En esa función lanzaremos una excepción que `try...catch` pueda capturar. Por ejemplo, supongamos una función que devuelve el inverso de un número (1/número). Si el número vale 0, se lanzará una excepción de división por cero, que podremos capturar con `try...catch`. Pero ¿qué ocurre si el error está en que nos equivocamos en el nombre de una variable?

Veamos el código:

```
<?php
function inverso($numero) {
    $numeroInverso=1/$numero;
    return $numeroInverso;
}
$numero=10;
try {
    $numeroInverso=inverso($numero);
    echo "El inverso de $numero es $numeroInverso<br/>";
}
catch(Throwable $excepcion) {
    echo "Se ha producido un error ".$excepcion->getMessage();
}
?>
```

Con el número 10 no hay problema; se mostrará que el inverso es 0,1.

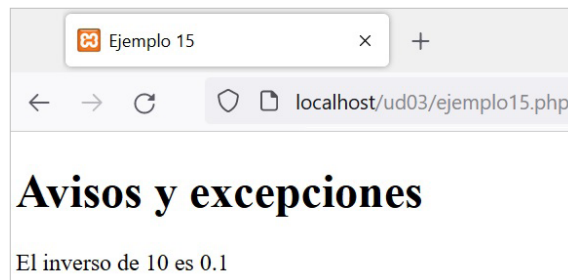


Figura 14.

Con el 0 tampoco hay problema; se captura la excepción y se mostrará el mensaje correspondiente:



Figura 15.

Ahora bien, si se nos cuela un error, como, por ejemplo, el nombre de una variable que no existe, como `$numreo` en lugar de `$numero`:

```
function inverso($numero) {
    $numeroInverso=1/$numreo;
    return $numeroInverso;
}
```

lo que ocurre es:



Figura 16.

Este tipo de errores no se pueden capturar como las excepciones. Como ya hemos comentado, en tiempo de desarrollo es mejor no desactivarlos, porque entonces no podríamos detectarlos. Es preferible hacer lo siguiente:

- » Crear una función propia de manejo de errores, que convierte cualquier error en excepción.

```
function manejaErrores($nivel, $mensaje, $fichero, $linea) {  
    throw new Exception("Error ".$mensaje." en la línea ".$linea." del fichero  
    ".$fichero);  
}
```

- » Sustituir el manejador de errores por defecto por el nuestro.

```
set_error_handler("manejaErrores");
```

- » Ahora el aviso será capturado por la función y se lanzará una excepción que será capturada por `try...catch`.



Figura 17.

- » En el momento en el que queramos volver al manejador de errores por defecto, lo podemos hacer con:

```
restore_error_handler();
```

Podemos guardar los errores en un fichero *log*, que consultaremos para conocer todos los detalles de cada error que se haya producido. Hay que tener en cuenta que el fichero de error no puede llamarse `error.log`, porque ya existe uno con ese nombre, en el que se guardan datos sobre posibles errores del servidor.

Usando funciones y datos del sistema como `get_current_user()`, para saber el usuario al que se le ha producido el error, o `$_SERVER['REMOTE_ADDR']` para saber desde qué máquina se ha lanzado el *script* que ha dado el error, podemos disponer de mucha información en nuestro registro de errores.

Una función típica de manejo de errores podría ser más o menos así:

```
function manejaErrores($eLevel, $eMessage, $eFile, $eLine){
    $mensaje = "Fecha: ".date("H:i d-m-Y ").$eMessage.
    " en el fichero ".$eFile." línea ".$eLine.
    " Usuario: ".get_current_user()." IP: ".
    $_SERVER['REMOTE_ADDR'];
    error_log("$mensaje in $eFile, line $eLine",
    3,
    "c:/xampp/apache/logs/user_errors");
}
```

Al usuario se le podrá mostrar un mensaje más o menos genérico («Póngase en contacto con su servicio técnico»), y para conocer más sobre el problema producido iremos al fichero `user_errors`.

05 Orientación a objetos con PHP

La programación orientada a objetos se basa en la definición de **clases**, a partir de las cuales se crean **objetos**. Cada clase tiene sus **atributos** y sus **métodos**, características a las que, en general, se accede a través de los objetos creados. Entre las clases pueden existir diversas relaciones, la más importante de las cuales es la de herencia, por la que una clase extiende a otra superior heredando sus atributos y métodos. Existen, por supuesto, variaciones a este modelo básico. A lo largo del apartado repasaremos los conceptos básicos y estudiaremos la forma de aplicarlos a PHP, así como algunas características novedosas propias del lenguaje.

Puedes acceder a los ficheros con todos los ejemplos de este apartado descargando el archivo `DWES_U03_A05.zip`, en la sección Ejemplos para trabajar y analizar de la unidad.

5.1. Clases, atributos y métodos en PHP

En PHP, como en otros lenguajes, una clase se define usando la palabra reservada **class**, seguida del nombre de la clase.

```
class Nombre
```

La clase puede incluir atributos y métodos. Ambos se definen después de crear la clase, entre llaves.

```
class Nombre {  
    ...//atributos  
    ...//método  
}
```

Las clases pueden extender a otras clases (herencia), o implementar interfaces previamente definidos. Para ello usamos, respectivamente, las palabras reservadas **extends** e **implements**.

```
class Nombre extends ClaseSuperior implements Interface {  
    ...//atributos  
    ...//métodos  
}
```

Los atributos pueden ser de tipo **public** (accesibles desde cualquier lugar), **protected** (accesibles desde la propia clase y clases derivadas) o **private** (accesibles solo desde dentro de la propia clase). Lo mismo se aplica a los métodos. El nombre de los atributos empieza también con el signo **\$**, como el resto de variables en PHP. Para el nombre de los métodos, se aplican las mismas condiciones que para el nombre de cualquier otra función. Los métodos van precedidos de la palabra reservada **function**. No es obligatorio declarar el tipo de dato que se devuelve, ni tampoco el tipo de cada parámetro. Dentro de los métodos, se puede hacer referencia a los propios atributos de la clase mediante el objeto genérico **\$this**, que hace referencia al propio objeto que esté en ese momento ejecutando el método, seguido de una flecha. Lo mismo puede aplicarse a métodos que son llamados desde otros métodos de la misma clase.

```
class Nombre extends NombreClaseSuperior implements NombreInterface {  
    private $atributo1;  
    protected $atributo2;  
    public function metodo1() {  
        return $this->atributo1;  
    }  
}
```


5.2. Creación y uso de objetos

Un objeto se crea asignándole un nombre y ejecutando el operador **new**, seguido del nombre de la clase.

```
$objeto = new Clase; // o bien $objeto = new Clase();
```

Para acceder a los atributos y métodos públicos se usa la flecha. Veamos un ejemplo completo:

```
<?php
class Persona {
    private $nombre;
    private $apellidos;
    public function getNombre() { return $this->nombre; }
    public function getApellidos() { return $this->apellidos; }
    public function setNombre($nombre) { $this->nombre=$nombre; }
    public function setApellidos($apellidos) {
$this->apellidos=$apellidos; }
}
$unaPersona=new Persona();
$unaPersona->setNombre("Jose Luis");
$unaPersona->setApellidos("González");
echo "Nombre completo: ".$unaPersona->getNombre()." ".$unaPersona-
>getApellidos();
?>
```

Los atributos son privados, y por tanto solo puede accederse a ellos, desde fuera de la clase, a través de los métodos públicos. Por ejemplo, si intentamos hacer algo como

```
$unaPersona->nombre="David"; // error
```

obtenemos un error **"Cannot Access private property"** (no puede accederse a un atributo privado).

Para saber si un objeto pertenece a cierta clase, podemos usar la instrucción de comparación **instanceOf**. Asimismo, disponemos de la función **get_class()**, que nos dice a qué clase pertenece un objeto.

```
if($unaPersona instanceof OtraClase) {
    echo "<br/>".$unaPersona->getNombre()." es otraclase";
}
else {
    echo "<br/>".$unaPersona->getNombre()." es de la clase ".get_
class($unaPersona);
}
```

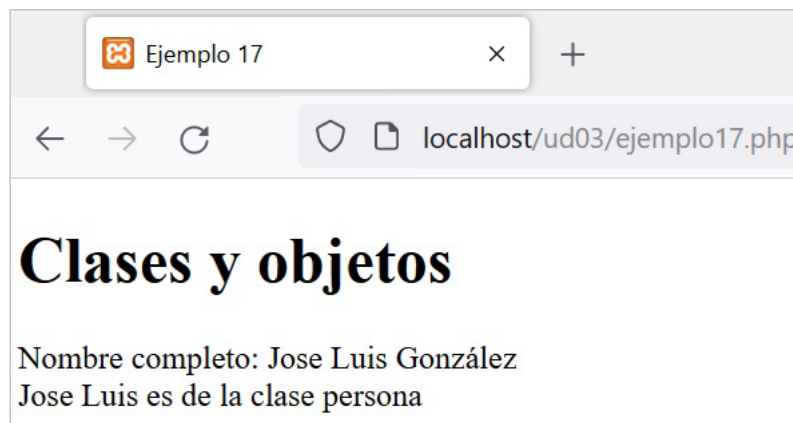


Figura 17.

Ejemplo: una clase rectángulo con una base y un altura en la que se puede calcular la superficie.

```
class Rectangulo {  
    private $base;  
    private $altura;  
    public function getBase() { return $this->base; }  
    public function getAltura() { return $this->altura; }  
    public function setBase($base) { $this->base=$base; }  
    public function setAltura($altura) { $this->altura=$altura; }  
    public function getSuperficie() { return $this->base * $this->altura; }  
}
```

Creamos un objeto de la clase **Rectangulo**, le asignamos una base y una altura y mostramos la superficie.

```
$rectangulo=new Rectangulo();  
$rectangulo->setBase(10);  
$rectangulo->setAltura(3);  
echo "El rectángulo de ".$rectangulo->getBase()." * ".$rectangulo->getAltura();  
echo " tiene una superficie de ".$rectangulo->getSuperficie();
```

5.3. Métodos constructores y destructores

A diferencia de otros lenguajes, el **método constructor** en PHP tiene un nombre concreto: `__construct()`. PHP también es diferente en que no permite definir más de un constructor para una misma clase. Así, si queremos tener un constructor con parámetros y otro sin parámetros, la solución es crearlo con parámetros que tengan valores por defecto, para el caso de que no se definan. El método constructor debe ser público.

Por ejemplo, en nuestro caso, podemos crear un constructor para que, si no se declaran la base y la altura al crear el objeto, se inicialicen por defecto a 1:

```
public function __construct($b=1, $a=1) {  
    $this->base=$b;  
    $this->altura=$a;  
}
```

Ahora vemos todas las opciones disponibles: pasar los valores por métodos `set`, no pasar ningún parámetro al constructor y trabajar con los valores por defecto, pasar ambos valores o pasar solo el primero.

```
$rectangulo=new Rectangulo();  
$rectangulo->setBase(10);  
$rectangulo->setAltura(3);  
echo "El rectángulo de ".$rectangulo->getBase()." * ".$rectangulo->getAltura();  
echo " tiene una superficie de ".$rectangulo->getSuperficie()."<br/>";  
$rectangulo2=new Rectangulo();  
echo "El rectángulo de ".$rectangulo2->getBase()." * ".$rectangulo2->getAltura();  
echo " tiene una superficie de ".$rectangulo2->getSuperficie()."<br/>";  
$rectangulo3=new Rectangulo(8,4);  
echo "El rectángulo de ".$rectangulo3->getBase()." * ".$rectangulo3->getAltura();  
echo " tiene una superficie de ".$rectangulo3->getSuperficie()."<br/>";  
$rectangulo4=new Rectangulo(8);  
echo "El rectángulo de ".$rectangulo4->getBase()." * ".$rectangulo4->getAltura();  
echo " tiene una superficie de ".$rectangulo4->getSuperficie()."<br/>";
```

Desde la versión PHP8 existe otra opción: podemos nombrar los parámetros cuyos valores pasamos. De esa forma, podemos pasarle un valor al segundo parámetro y dejar que el primero asuma su valor por defecto.

```
// SOLO A PARTIR DE PHP 8.0  
$rectangulo5=new Rectangulo(a: 9);  
echo "El rectángulo de ".$rectangulo5->getBase()." * ".$rectangulo5->getAltura();  
echo " tiene una superficie de ".$rectangulo5->getSuperficie()."<br/>";
```

PHP dispone también de un método destructor, `__destruct()`. Este método debe ser público y se llama automáticamente cada vez que desaparecen todas las referencias a un objeto determinado o si salimos de la aplicación. En su interior escribiremos las instrucciones que queremos ejecutar cuando eso ocurra.

No podemos llamar directamente al destructor, pero podemos forzar la desaparición de las referencias a un objeto con `unset($objeto)`, lo que provoca la ejecución del método destructor de la clase correspondiente.

5.4. Herencia

La herencia es una propiedad de las clases por la cual unas pueden extender a otras, absorbiendo y ampliando sus propiedades y métodos. Una clase (derivada) hereda de otra clase (principal) usando la palabra reservada **extends** en su declaración. La clase derivada tiene acceso a las propiedades y métodos (salvo los que fueran privados) de la clase principal, además de tener los suyos propios. La clase derivada también tiene la facultad de sobrescribir los métodos heredados de la clase principal. Veamos un ejemplo con nuestra clase **Rectangulo**, que hemos mostrado anteriormente:

La clase Cuadrado tiene una base, y a partir de ella calculamos su superficie.

```
class Cuadrado {
    protected $base;
    public function getBase() {
        return $this->base;
    }
    public function
    setBase($base) {
        $this->base=$base;
    }
    public function
    getSuperficie() {
        return $this->base *
        $this->base;
    }
}
```

La clase Rectangulo hereda la propiedad base de Cuadrado y añade la propiedad altura. También hereda el método getSuperficie(), pero como su superficie se calcula de manera diferente, lo sobrescribe.

```
class Rectangulo extends Cuadrado {
    private $altura;
    public function getAltura() {
        return $this->altura;
    }
    public function setAltura($altura) {
        $this->altura=$altura;
    }
    public function getSuperficie() {
        return $this->base * $this-
        >altura;
    }
}
```

Esquema 7.

En PHP, una clase solo puede extender a otra clase, que debe haberse definido antes que la clase derivada.

Si la clase principal dispone de un método constructor, la clase derivada también lo hereda, y se invocará automáticamente al crearse un objeto de cualquiera de las dos clases. Ahora bien, si la clase derivada dispone de su propio constructor, entonces, cuando se cree un objeto de una de las dos clases, solo se invocará el método constructor propio. Si desde una

clase derivada con constructor propio queremos llamar al constructor de la clase principal, lo podemos hacer mediante **parent** y el operador de resolución de ámbito **::**, operador que permite llamar a cualquier método de la clase principal que haya sido sobrescrito en la clase derivada.

Por ejemplo, dada una clase **Persona**:

```
class Persona {  
    protected $nombre;  
    protected $apellidos;  
    protected $domicilio;  
    protected $telefono;  
    function __construct($nombre,$apellidos,$domicilio,$telefono) {  
        $this->nombre=$nombre;  
        $this->apellidos=$apellidos;  
        $this->domicilio=$domicilio;  
        $this->telefono=$telefono;  
    }  
}
```

Podemos derivar de ella una clase **Alumnado**, que tenga los mismos atributos, además del número de matrícula. Cuando se cree un objeto de la clase **Alumnado** se invocará su método constructor, que a su vez llamará al método constructor de la clase **Persona** y posteriormente guardará el valor del atributo específico de **Alumnado**; en este caso, el número de matrícula.

```
class Alumnado extends Persona {  
    private $numeroMatricula;  
    function __construct($nombre,$apellidos,$domicilio,$telefono,  
$numMatricula) {  
        parent::__construct($nombre,$apellidos,$domicilio,$telefono);  
        $this->numeroMatricula=$numMatricula;  
    }  
}
```

El operador **::** permite realizar otras muchas operaciones. Puedes encontrar más información en uno de los enlaces incluidos en el material complementario.

5.5. Los modificadores final, const y static

El modificador **final**, aplicado a una clase, impide que se puedan crear clases derivadas de ella. El mismo modificador, aplicado a un método, permite que sea heredado por las clases derivadas, pero no sobrescrito.

<pre>final class ClaseNoExtensible { ... }</pre>	<pre>class Extensible { final function noExtensible() { ... // la clase puede ser extendida por otra // pero este método no podrá ser sobrescrito } }</pre>
--	---

Esquema 8.

El modificador **const** permite definir un atributo constante dentro de una clase. Dado que todos los objetos que se creen de dicha clase compartirán el valor del atributo, este podrá ser accedido tanto desde cualquier objeto como simplemente indicando el nombre de la clase. Los nombres de los atributos constantes diferencian entre las mayúsculas y las minúsculas. Así, las constantes **Iva** e **IVA** no serían la misma.

<pre>class Producto { const IVA=21; }</pre>	<pre>\$articulo=new Producto; echo \$articulo::IVA; // acceso desde el objeto echo Producto::IVA; // acceso desde la clase</pre>
---	--

Esquema 9.

El operador **static** permite definir atributos y métodos de clase, que pueden llamarse sin necesidad de crear objetos. En el caso de los atributos, hay una diferencia respecto a los definidos como constantes: si se definen como **static**, su valor sí que se puede modificar. Eso sí, se modifica para todos los objetos de la clase, los existentes y los futuros. Un atributo **static** no puede accederse desde dentro de la clase con **\$this**. Usaremos **self** para hacerlo desde la propia clase, y **parent** para hacerlo desde las clases derivadas. Veamos un ejemplo:

```
class Producto {
    public static $IVA=21;
    public static function getIVA() { return self::$IVA; }
    public static function setIVA($iva) { self::$IVA = $iva; }
}
```

El atributo **\$IVA** es estático, lo que significa que su valor es el mismo para todos los objetos que se creen de la clase **Producto**. Además puede ser accedido, tanto directamente como a través de sus métodos **set** y **get** (que también son estáticos), sin tener que crear ningún objeto. Si hacemos

```
echo "El IVA genérico es del ".Producto::getIVA()."%.<br/>";
$articulo1=new Producto();
echo "IVA del producto 01: ".$articulo1->getIVA()."%.<br/>";
echo "Cambiamos el valor general del IVA<br/>";
Producto::setIVA(22);
```

```
$articulo2=new Producto();  
echo "IVA del producto 02: ".$articulo2->getIVA()."%<br/>";  
echo "IVA del producto 01: ".$articulo1->getIVA()."%<br/>";
```

obtendremos:

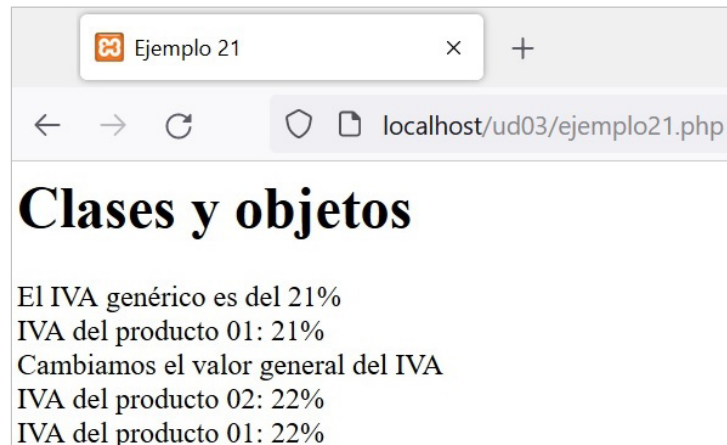


Figura 18.

Puedes encontrar información complementaria relacionada con estos apartados en la sección Documentos de descarga, en los siguientes archivos:

DWES_U03_MC_Interfaces, clases abstractas y traits.

DWES_U03_MC_Clases anónimas, funciones anónimas y objetos de cierre.

DWES_U03_MC_Espacios de nombres.



VÍDEOLAB

Para consultar el siguiente vídeo sobre página Login, escanea el código QR o [pulsa aquí.](#)

2 CASO PRÁCTICO



IMPLEMENTAMOS UNA CUENTA BANCARIA

Vamos a crear una clase sencilla que implemente el funcionamiento básico de una cuenta bancaria. Para ello, deberás crear una clase **CuentaBancaria**, con los siguientes atributos privados: **usuario**, **pin**, **entradas**, **salidas** y **saldo**. Los atributos **entradas** y **salidas** serán *arrays* en los que se irán guardando las cantidades que se van ingresando o sacando de la cuenta. El saldo será la cantidad que hay en la cuenta. Al crear un objeto de la clase **CuentaBancaria** se deberán pasar al constructor el nombre del usuario y el pin, que se guardarán en el objeto. Además, se inicializarán los *arrays* **entrada** y **salida**, y se pondrá el **saldo** a 0.

Además del método constructor, la clase debe incluir los siguientes métodos:

- » Un método **cambiaPin**, que permita cambiar el pin del usuario.
- » Un método **validaUsuario**, al que se le pase un nombre de usuario y un pin y valide si es correcto (devolvería **true**) o no lo es (devolvería **false**).
- » Un método **ingresar**, al que se le pase una cantidad, la añada al *array* de entradas, y además incremente el saldo de la cuenta en la cantidad indicada.
- » Un método **sacar**, al que se le pase una cantidad y, si es menor o igual que el saldo, añada la cantidad al *array* de salidas y la reste del saldo. En este caso, el método devolverá **true**. Si la cantidad que se desea sacar de la cuenta es superior al saldo, el método no hará nada y devolverá **false**.
- » Métodos **getSaldo()**, **getEntradas()** y **getSalidas()** para devolver, respectivamente, el saldo, la lista de entradas y la lista de salidas.

SOLUCIÓN

A continuación, puedes ver el código fuente de la clase solicitada.


```

class CuentaBancaria {
    private $usuario;
    private $pin;
    private $entradas;
    private $salidas;
    private $saldo;
    public function __construct($usuario, $pin) {
        $this->usuario=$usuario;
        $this->pin=$pin;
        $this->entradas=array();
        $this->salidas=array();
        $this->saldo=0;
    }
    public function cambiaPin($pin) {
        $this->pin=$pin;
    }
    public function validaUsuario($usuario, $pin) {
        if($this->usuario==$usuario && $this->pin==$pin) {
            return true;
        }
        else {
            return false;
        }
    }

    public function ingresar($cantidad) {
        $this->entradas[]=$cantidad;
        $this->saldo=$this->saldo+$cantidad;
    }
    public function sacar($cantidad) {
        if($cantidad > $this->saldo) {
            return false;
        }
        else {
            $this->salidas[]=$cantidad;
            $this->saldo=$this->saldo-$cantidad;
            return true;
        }
    }
    public function getSaldo() {
        return $this->saldo;
    }
    public function getEntradas() {
        return $this->entradas;
    }
    public function getSalidas() {
        return $this->salidas;
    }
}

```

Dicho código, junto con un ejemplo de funcionamiento de la clase, está disponible también en el archivo **DWES_U03_CP_A05_SOL.zip**, en la parte inferior izquierda.

UAX FP